

Explanation of Logic

Detailed Rationale Behind Every Query & Design Decision

Team: Event Horizon

Competition: Data Vortex 2026

Phase 2: SQL Analysis & Insight Generation

1. Schema Design Rationale

1.1 Why a Relational Schema in SQLite?

We designed a strict relational schema (star-schema equivalent) loaded into SQLite. We preserved all original data anomalies from Phase 1 without fabricating or clipping data during the load process. Analytical filtering is handled dynamically via SQL WHERE clauses rather than destructive ETL transforms.

1.2 Location Splitting (city, country)

The original CSV had a single 'location' column with values like 'Munich, Germany'. We split this into separate 'city' and 'country' columns during the data load to enable granular multi-level geographic grouping in M1 without runtime string manipulation.

1.3 Imputation Flags Retained

Columns like `location_was_null`, `language_was_null`, `account_created_was_null`, and `platform_was_null` are boolean flags carried from Phase 1. They record which values were originally missing and later imputed so that downstream analysis can distinguish real values from synthetic ones.

1.4 The `v_post_engagement` View - No COALESCE

We created a virtual view (`v_post_engagement`) that computes `total_engagement = likes + shares + comments`. Critically, we do NOT use COALESCE to substitute zeros for NULLs. If any engagement component is NULL, `total_engagement` will also be NULL. This is the truthful representation of 'unknown'. Queries filter explicitly using `WHERE has_null_component = 0` or `WHERE likes >= 0` rather than fabricating data.

2. E3 Logic - Average Engagement by Platform

2.1 Filters Applied

WHERE platform IS NOT NULL AND likes >= 0

Two filters are applied:

- platform IS NOT NULL: 1,784 rows dropped. A platform average cannot include posts without a platform assignment.
- likes >= 0: An additional 421 rows dropped. Negative likes are imputation artifacts from Phase 1 (not 'dislikes') and physically distort aggregations.

2.2 Total Dataset Evaluated

After filtering: 9,795 posts across 5 platforms (YouTube, Instagram, Facebook, Reddit, Twitter).

2.3 Why AVG and Not SUM?

The question asks for 'average engagement'. AVG normalizes by post count, making comparison fair across platforms with different volumes. YouTube has 1,990 posts while Instagram has 1,893 - raw sums would bias toward higher-volume platforms.

2.4 Outcome

YouTube generates the highest average total engagement at 4,026.89 per post.

3. M1 Logic - Location Engagement Ranking

3.1 Filters Applied

WHERE p.likes >= 0

Only negative likes are filtered (509 rows). We intentionally retain posts with NULL platforms because they still represent valid geographic engagement - a post from Munich is still from Munich regardless of whether its platform field was imputed.

3.2 JOIN Strategy

INNER JOIN between users and v_post_engagement on user_id. This naturally excludes any orphan posts (posts without a matching user), though our validation confirmed zero orphans exist in this dataset.

3.3 Grouping by City + Country

We group by both city AND country to handle potential city name collisions (e.g., 'Portland, USA' vs a hypothetical 'Portland, UK'). The location was split during load specifically to enable this granularity.

3.4 LIMIT 10

The question asks to 'rank locations from highest to lowest'. We apply LIMIT 10 to surface the top performers. The full ranked list is available by removing the LIMIT clause.

3.5 Outcome

Munich, Germany ranks #1 with 1,753,727 total engagement across 439 posts (avg 3,994.82 per post). Geographic engagement is not tied solely to English-speaking hubs.

4. H3 Logic - Platform Performance vs Own Average

4.1 The Core Problem

The question asks: 'Identify posts whose engagement is significantly higher ($>2x$) than the average engagement of that platform.' The literal query returns 0 rows. This is NOT a bug - it is the correct mathematical result.

4.2 Part 1: Diagnostic Proof

We first run a diagnostic query that computes each platform's avg engagement, the $2x$ threshold, and the actual maximum engagement. Result: no platform's maximum reaches $2x$.

For example, YouTube's avg is 4,026.9, so $2x = 8,053.8$, but YouTube's maximum post is only 7,755 (ratio 1.926). This holds across all 5 platforms.

4.3 Statistical Justification

The dataset uses a bounded uniform distribution: Likes max 5000, Shares max 2000, Comments max 1000. The absolute maximum possible engagement is 8,000. Because the mean across platforms sits near $\sim 4,000$, it is mathematically impossible for any value to reach $2x$ the mean. $\max(8000) < 2 * \text{mean}(\sim 4000) = \sim 8000$. The distributions are too tight.

4.4 Part 2: Empty Result (Honest Answer)

The literal query using `WHERE total_eng >= 2 * platform_avg_eng` correctly returns 0 rows. We present this as evidence rather than hiding it.

4.5 Part 3: Meaningful Alternative

Since no post reaches $2x$, we answer the question's TRUE INTENT by identifying posts that most outperform their own platform. We use `PERCENT_RANK()` to find the top 1% (99th percentile) performers relative to their platform, yielding ~ 100 posts with ratios ranging from 1.773 to 1.962.

4.6 Business Interpretation

The near-uniform engagement across posts implies the platform's distribution mechanics are not producing viral outliers. Growth must come from volume, not from chasing unicorn viral posts. This is itself a meaningful finding about the data generation process.

5. Bonus Queries Logic

5.1 Bonus 1: MoM Growth (Trend Detection)

We use `strftime('%Y-%m', timestamp)` to group posts by calendar month per platform, then apply the `LAG()` window function to compute month-over-month percentage growth.

`LAG(monthly_engagement) OVER (PARTITION BY platform ORDER BY month)` retrieves the previous month's engagement for the same platform. The growth formula is:

$$\text{mom_growth_pct} = (\text{current} - \text{previous}) * 100.0 / \text{NULLIF}(\text{previous}, 0)$$

`NULLIF` prevents division by zero. The first month per platform naturally returns `NULL` for growth (no prior month to compare against).

5.2 Bonus 2: Anomaly Discovery (H5)

Leveraging Phase 1 audit knowledge, we use a `CASE` expression to classify specific anomaly types:

- Negative Likes: Imputation artifacts where missing values were filled with negative numbers during Phase 1 data cleaning.
- Missing Platform: 1,784 posts with `NULL` platform field.
- HTML/Entity Corruption: Posts where `text_content` contains encoded HTML entities like `&` or malformed tags.

The `WHERE` clause uses `OR` to capture all anomaly types in a single pass. `LIMIT 20` constrains output for readability.

6. Validation Logic

6.1 Purpose

The validation script proves data integrity after the CSV-to-SQLite load. It uses UNION ALL to combine multiple aggregate checks into a single result set.

6.2 Checks Performed

1. users_count (1,501): Total users loaded matches raw CSV row count.
2. posts_count (12,000): Total posts loaded matches raw CSV row count.
3. duplicate_users (0): No duplicate user_ids - PRIMARY KEY constraint held.
4. duplicate_posts (0): No duplicate post_ids - PRIMARY KEY constraint held.
5. null_platforms (1,784): Known missing platforms preserved (not dropped).
6. negative_likes (509): Known imputation artifacts preserved (not corrected).
7. max_shares (2,000): Upper bound matches expected data generation ceiling.
8. min_shares (0): Lower bound is zero, confirming no negative shares.

6.3 Design Choice

String literals (e.g., 'users_count') are ROW LABELS only. Every value is computed at runtime by an aggregate over a real table. No result in this validation is hardcoded.